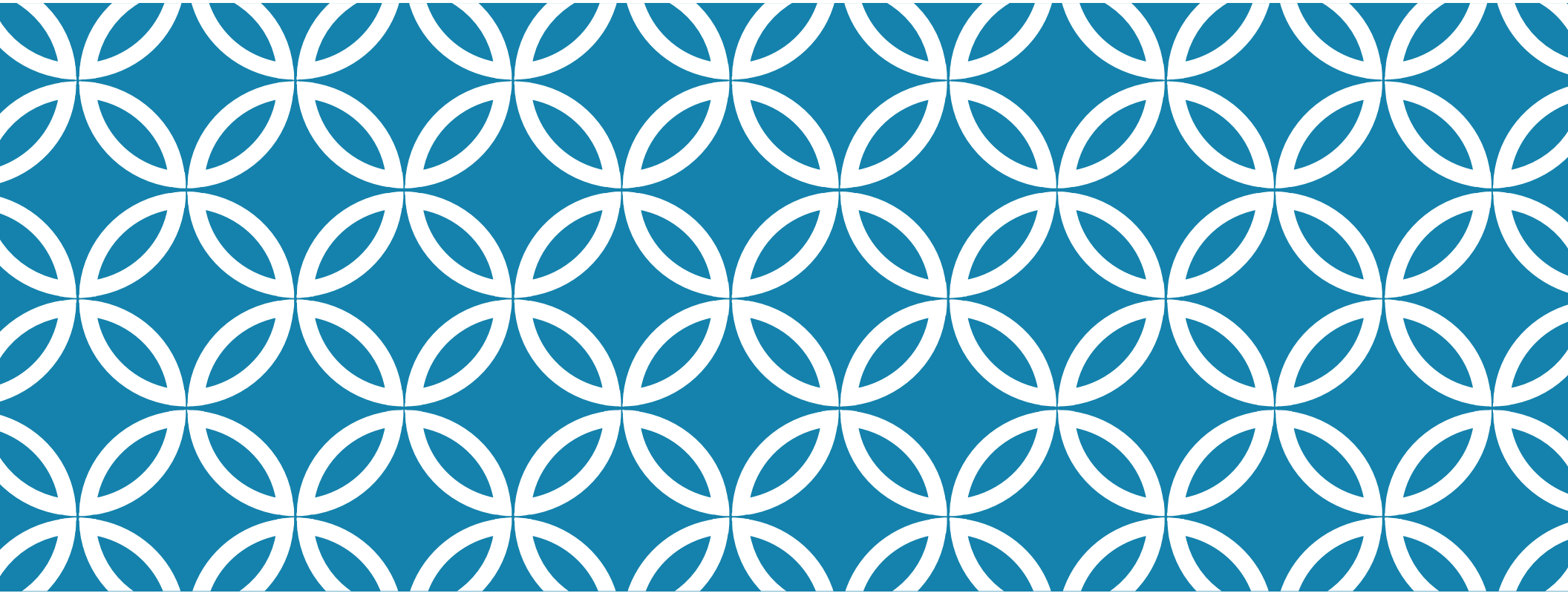
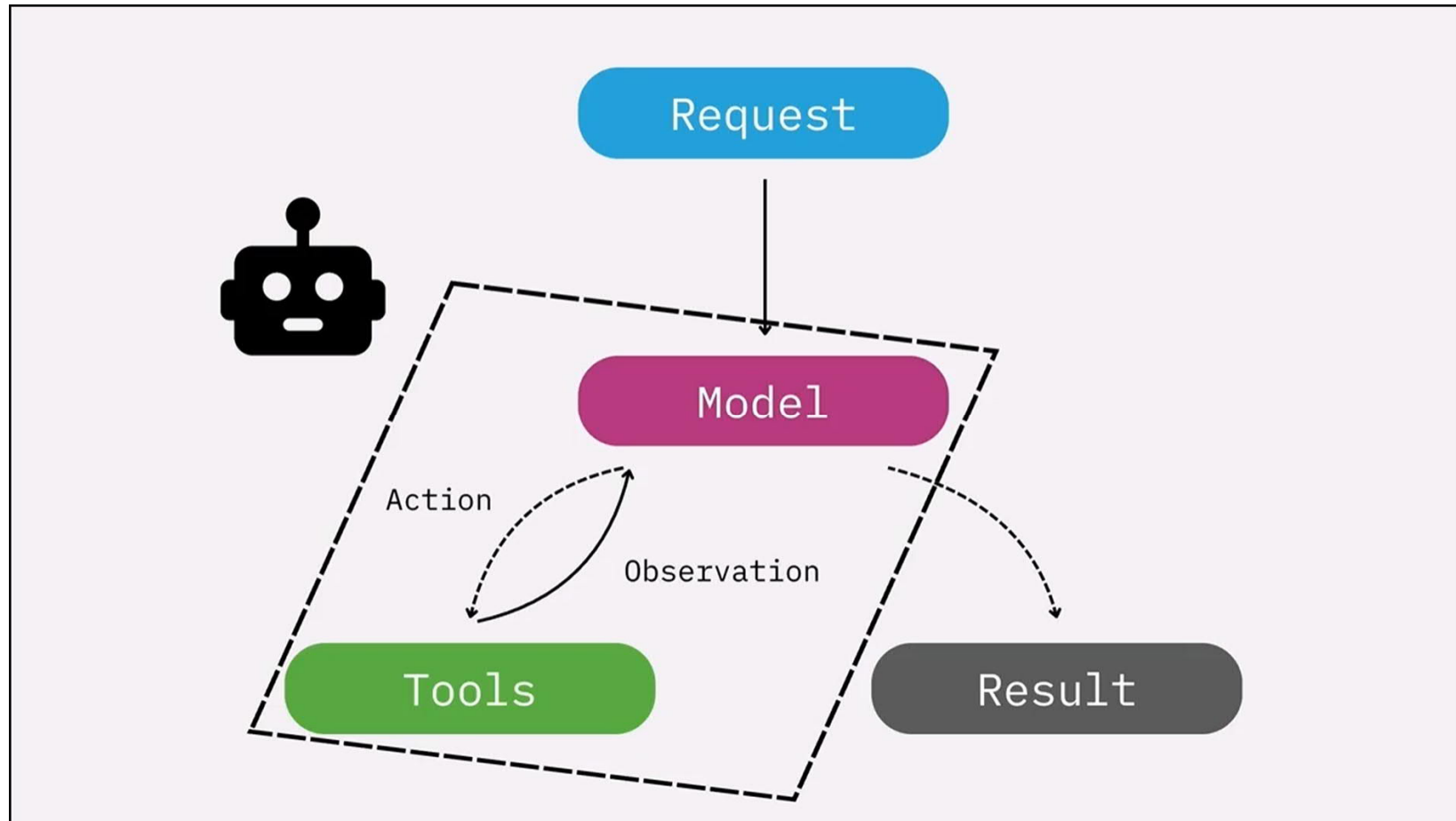




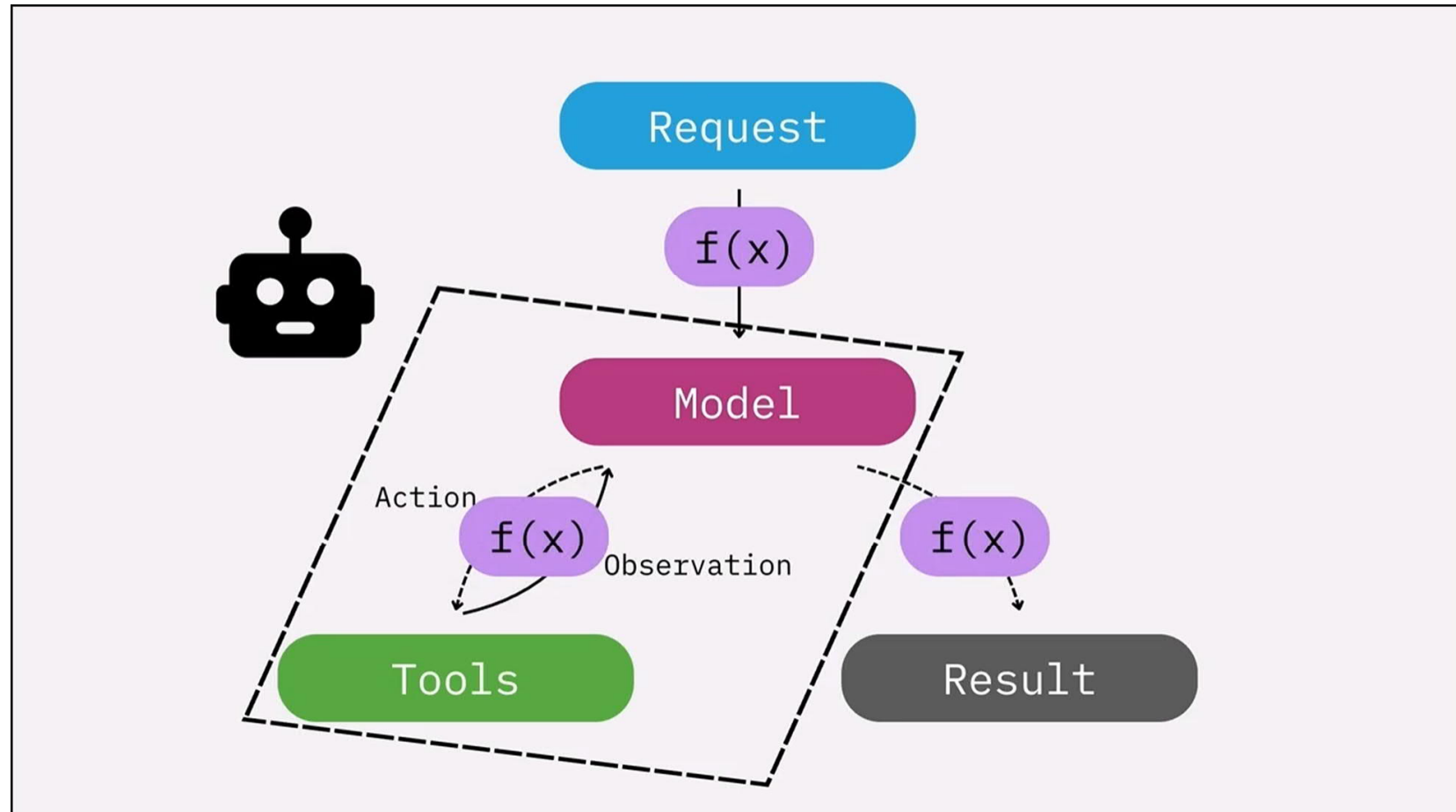
AGENTS — BEYOND THE BASICS

CS5305

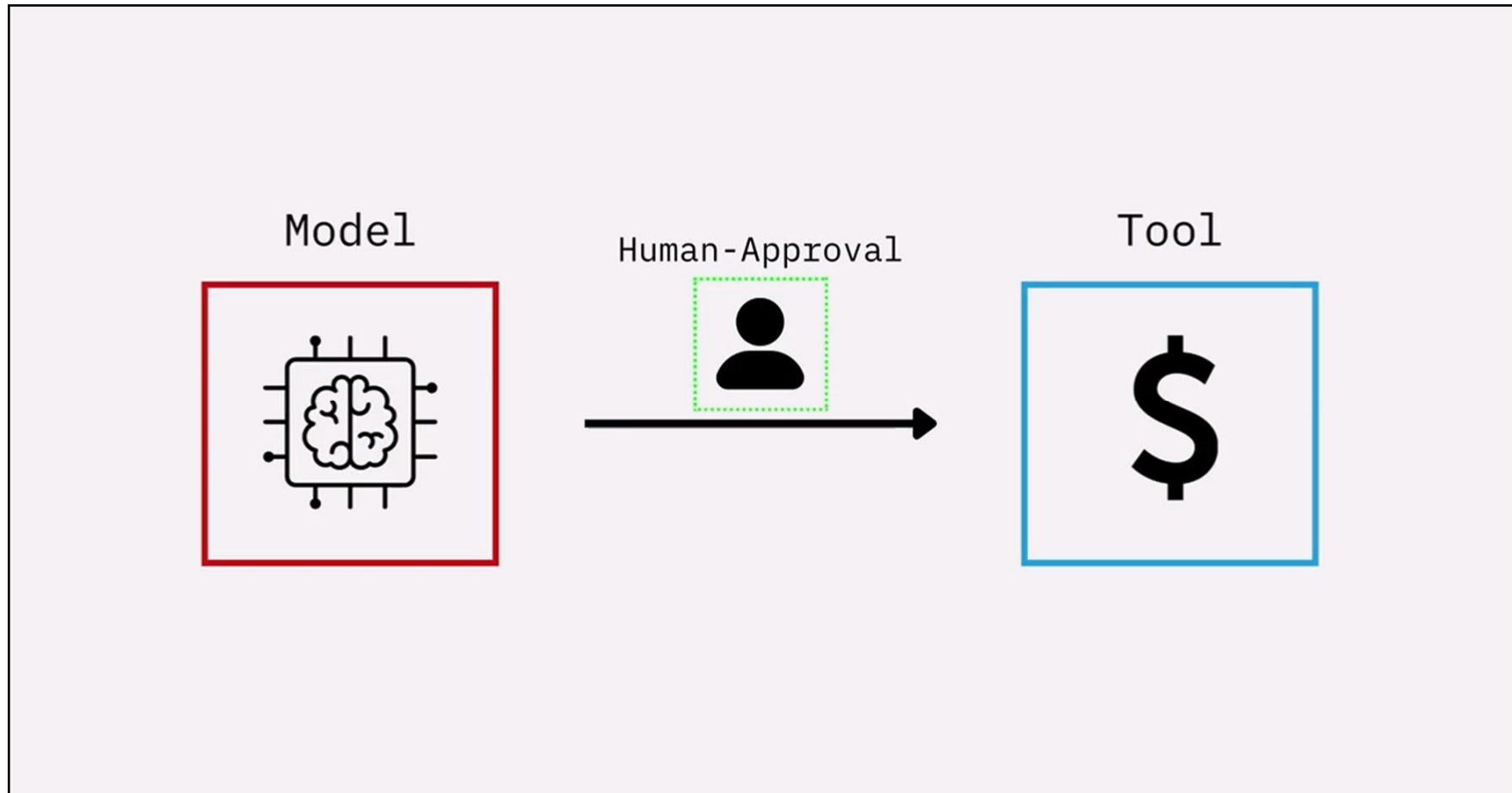
PREVIOUSLY...



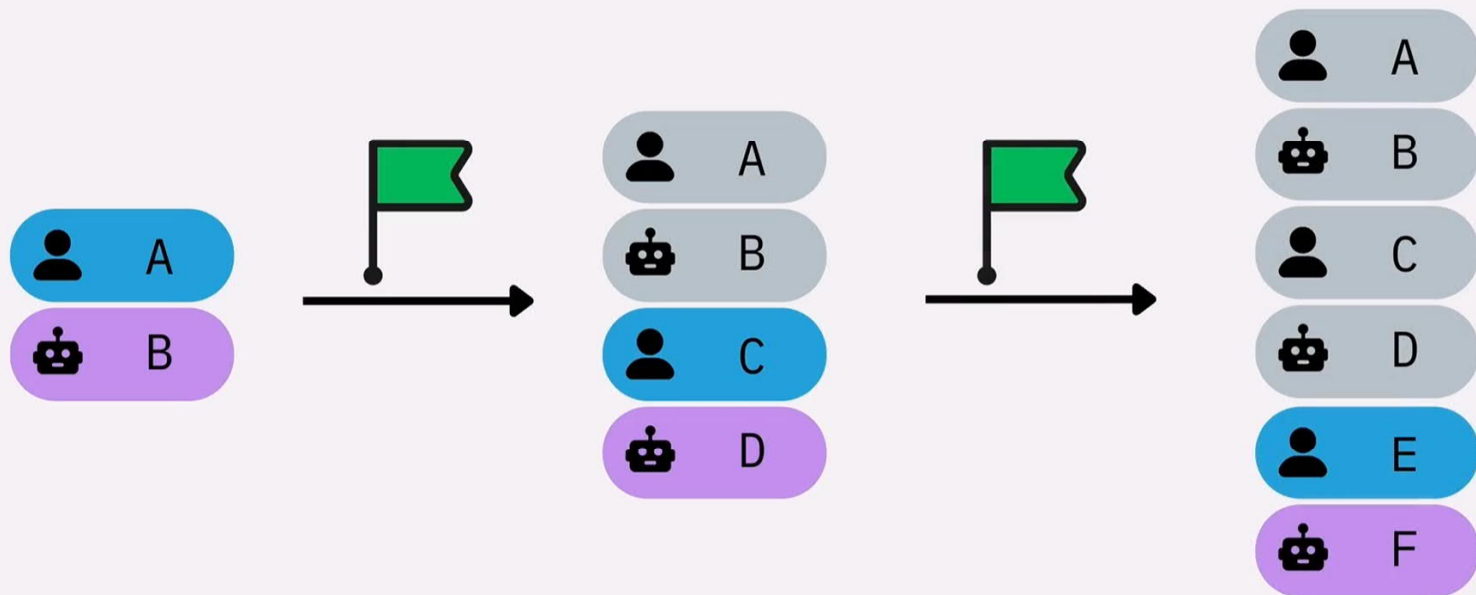
MIDDLEWARE — DYNAMIC RESOURCE SELECTION



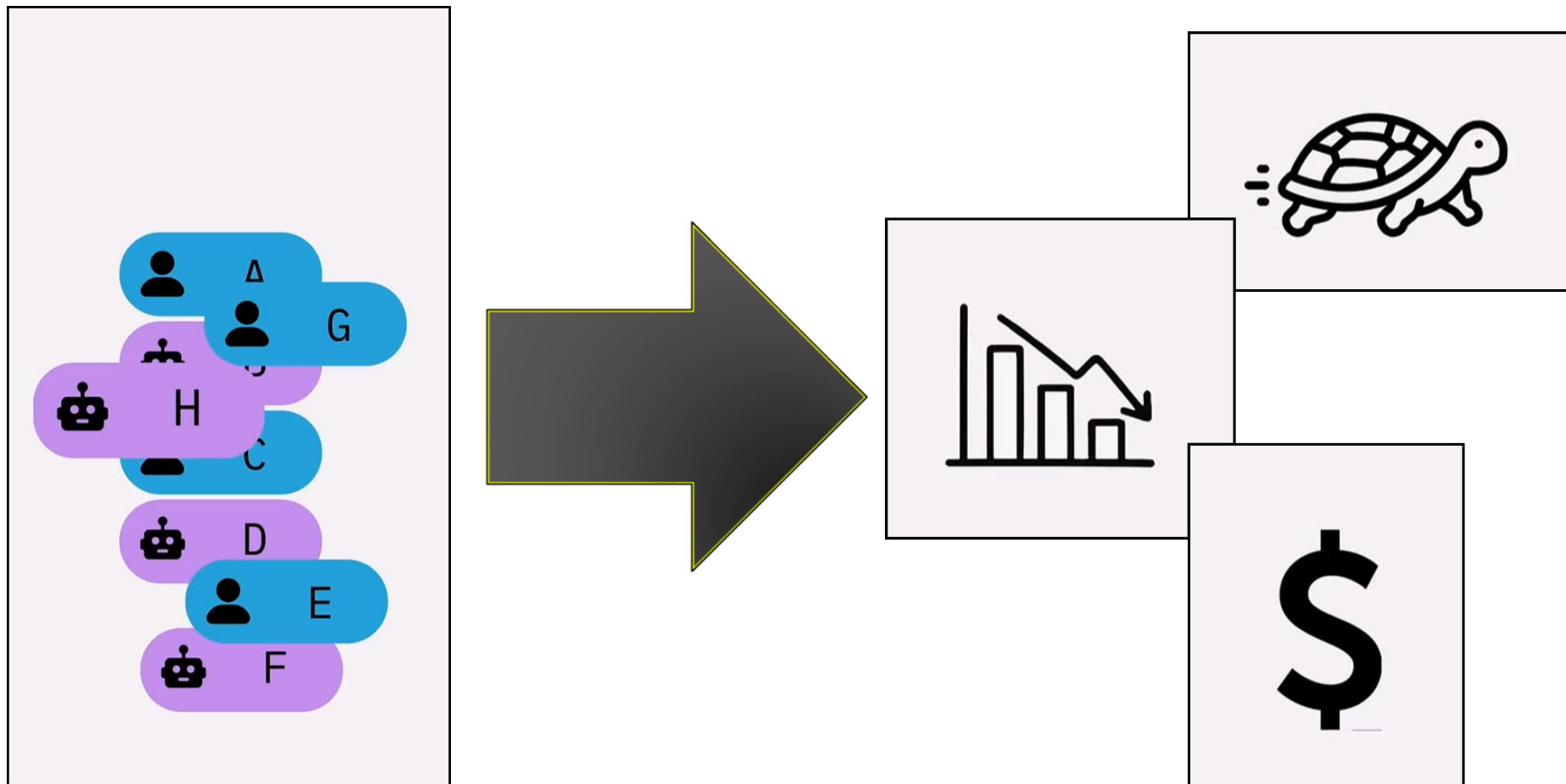
EXAMPLE: HUMAN IN THE LOOP

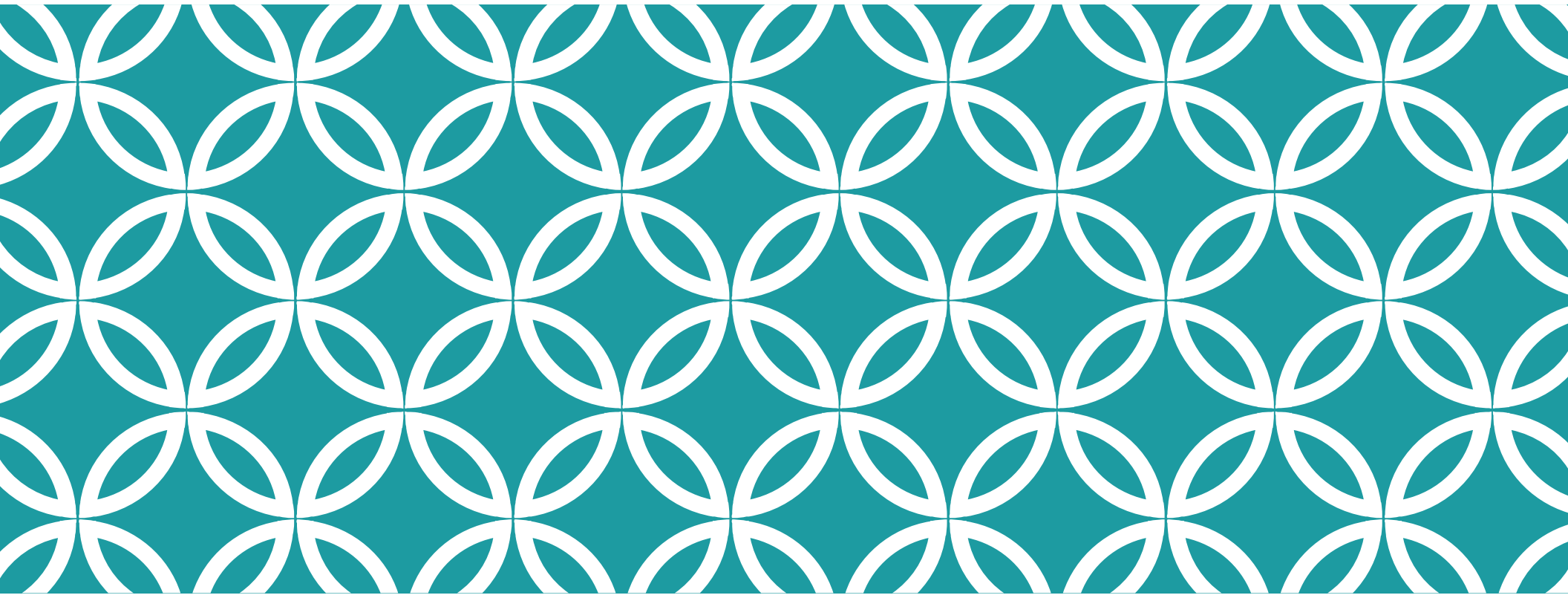


MANAGING LONG CONVERSATIONS



STANDARD REQ/RESP NOT FEASIBLE WITH LLMS!





MIDDLEWARE |

MIDDLEWARE

Middleware API in langchain allows the developer to control the internal workings of the the agent.

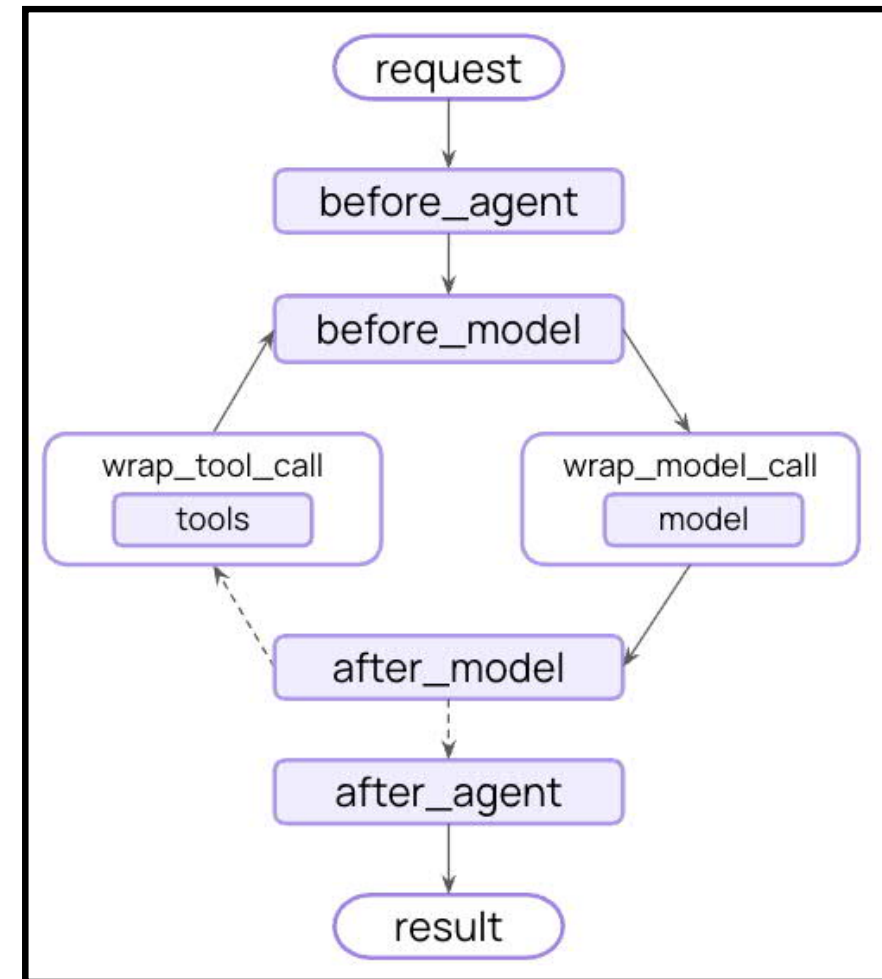
Middleware is useful for the following:

- Tracking agent behavior with logging, analytics, and debugging.
- Transforming prompts, tool selection, and output formatting.
- Adding retries, fallbacks, and early termination logic.
- Applying rate limits, guardrails, and PII detection.
 - Guardrails help you build safe, compliant AI applications by validating and filtering content at key points in your agent's execution.
 - They can detect sensitive information, enforce content policies, validate outputs, and prevent unsafe behaviors before they cause problems.

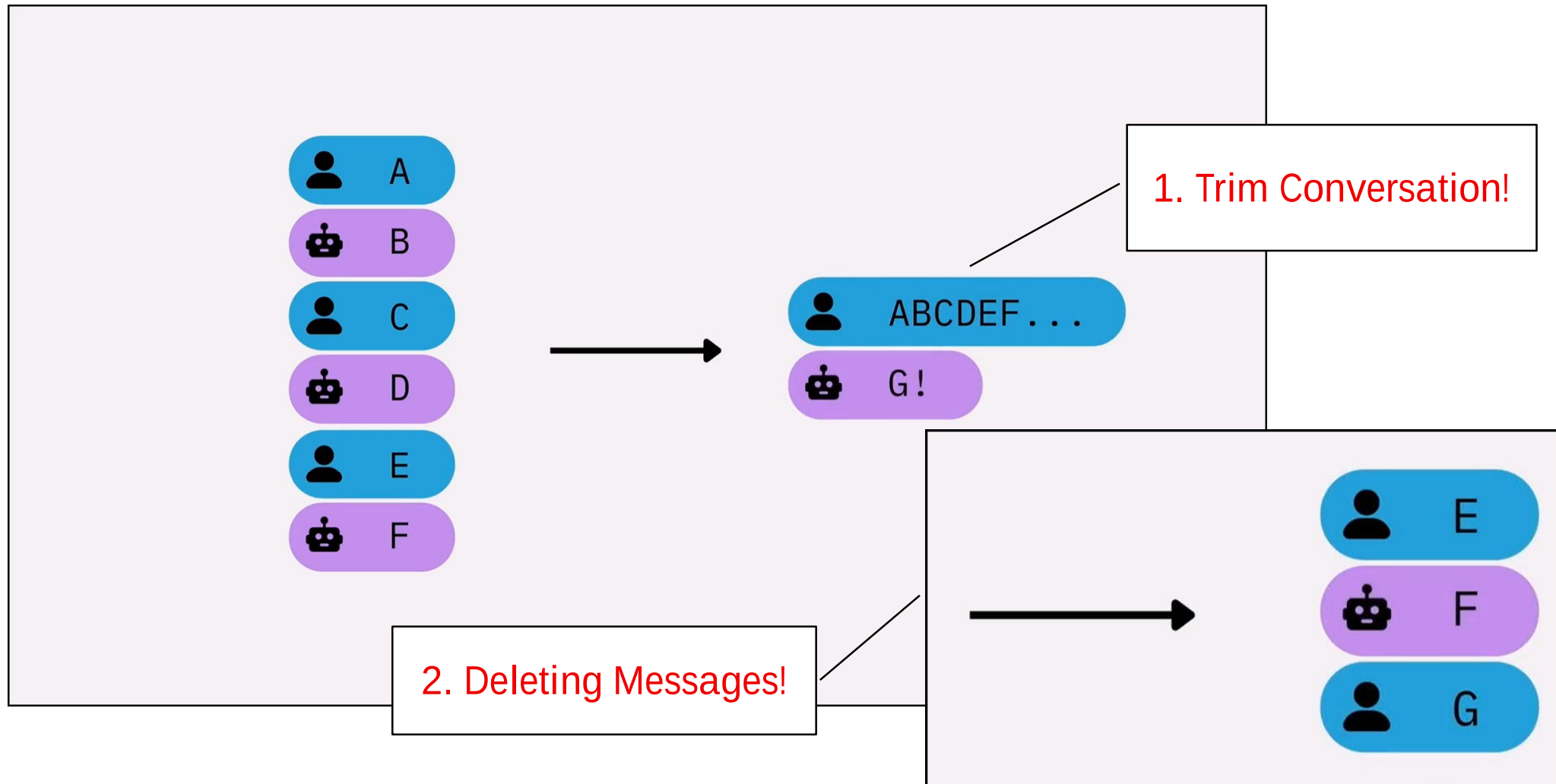
MIDDLEWARE (CONT...)

- Middleware exposes hooks before and after each of these steps inside the agent.
- Langchain comes with an extensive list of pre-built middleware for common use cases.

Summarization	Automatically summarize conversation history when approaching token limits.
Human-in-the-loop	Pause execution for human approval of tool calls.
File search	Provide Glob and Grep search tools over filesystem files.



EXAMPLE: MANAGING LONG CHATS



MANAGING LONG CHATS

```
from langchain.agents import create_agent
from langchain.agents.middleware
    import SummarizationMiddleware, HumanInTheLoopMiddleware

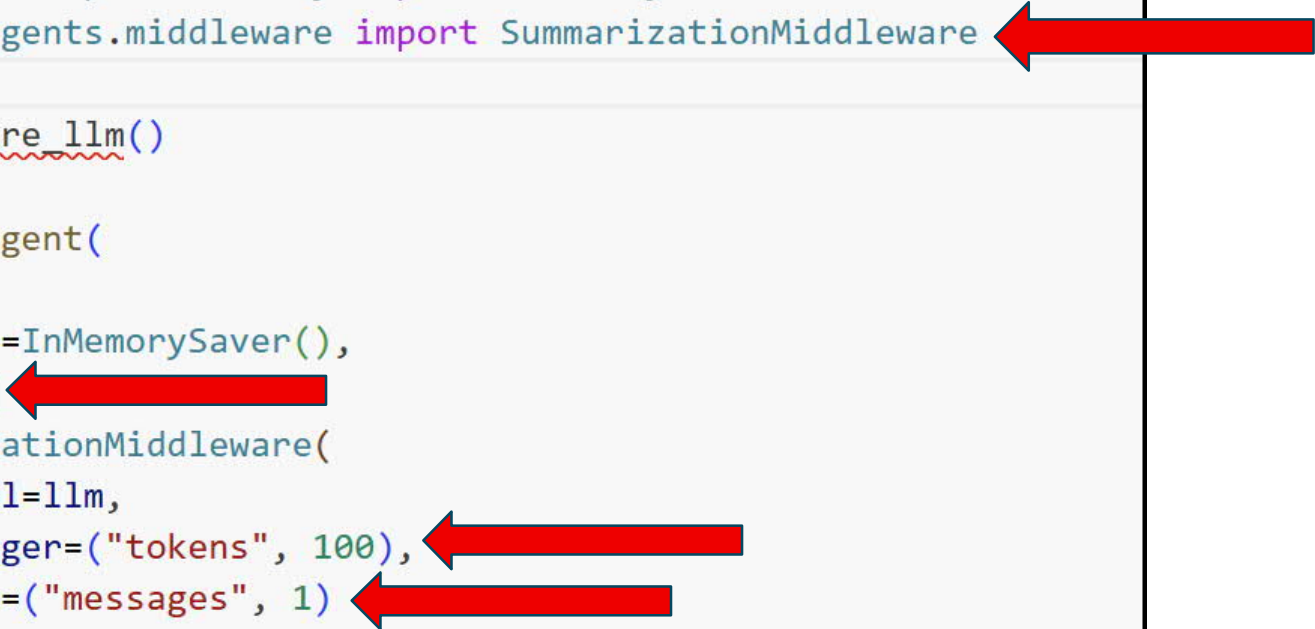
agent = create_agent(
    model="gpt-4.1",
    tools=[...],
    middleware=[
        SummarizationMiddleware(...),
        HumanInTheLoopMiddleware(...)
    ],
)
```

SUMMARIZATION MIDDLEWARE

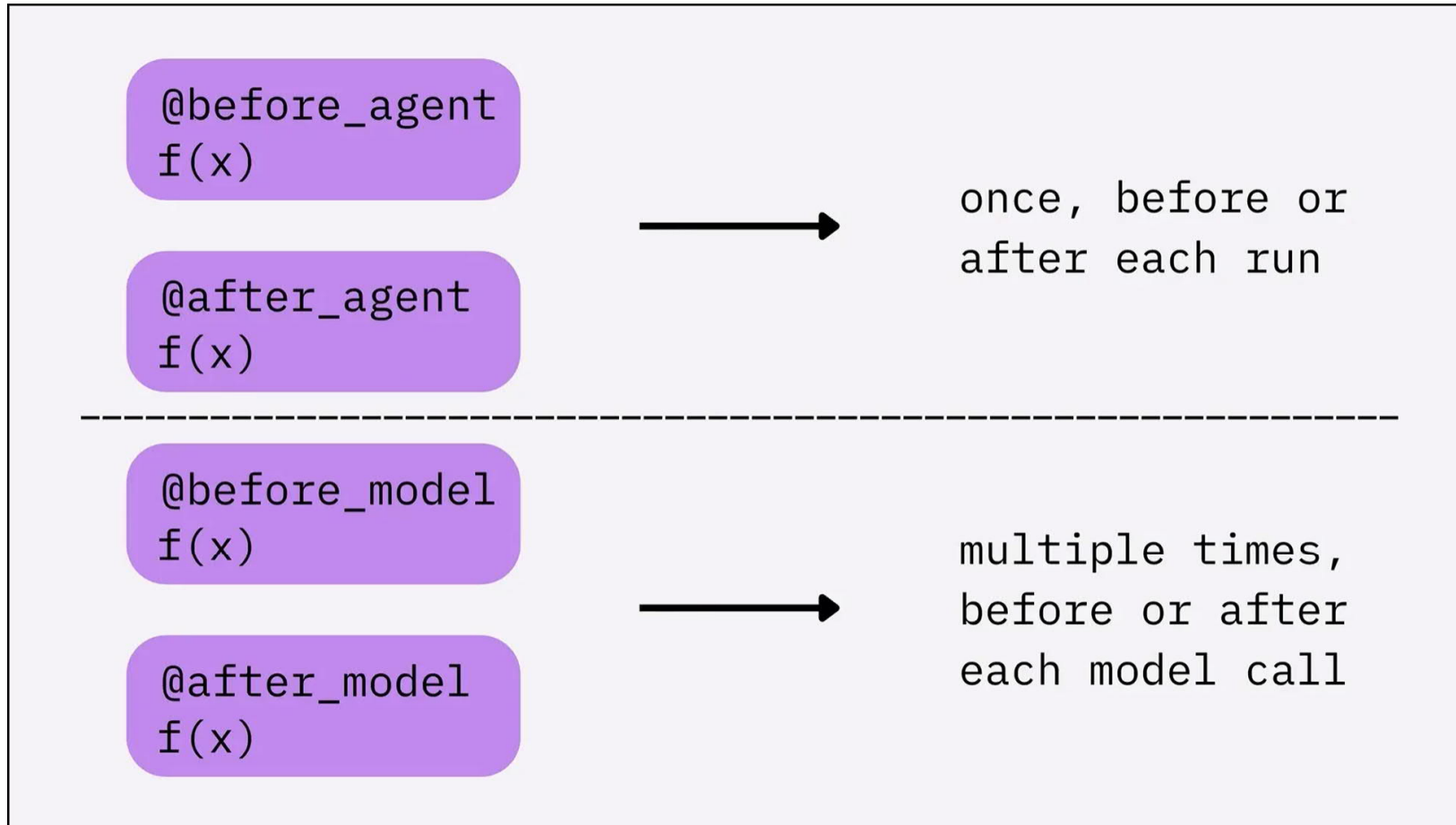
```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import SummarizationMiddleware

llm = create_azure_llm()

agent = create_agent(
    model=llm,
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(
            model=llm,
            trigger=("tokens", 100),
            keep=("messages", 1)
        )
    ]
)
```



CONTROLLING THE REACT LOOP THROUGH MIDDLEWARE



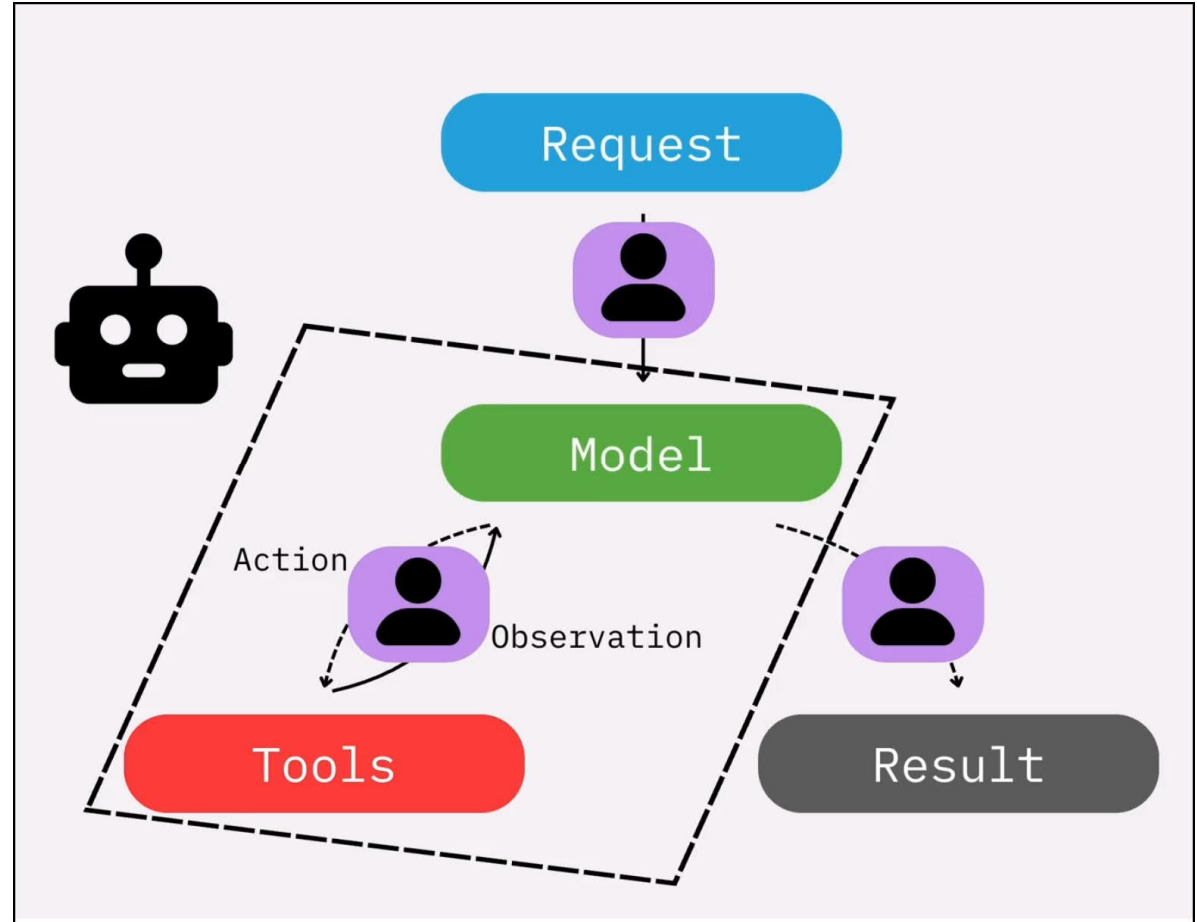
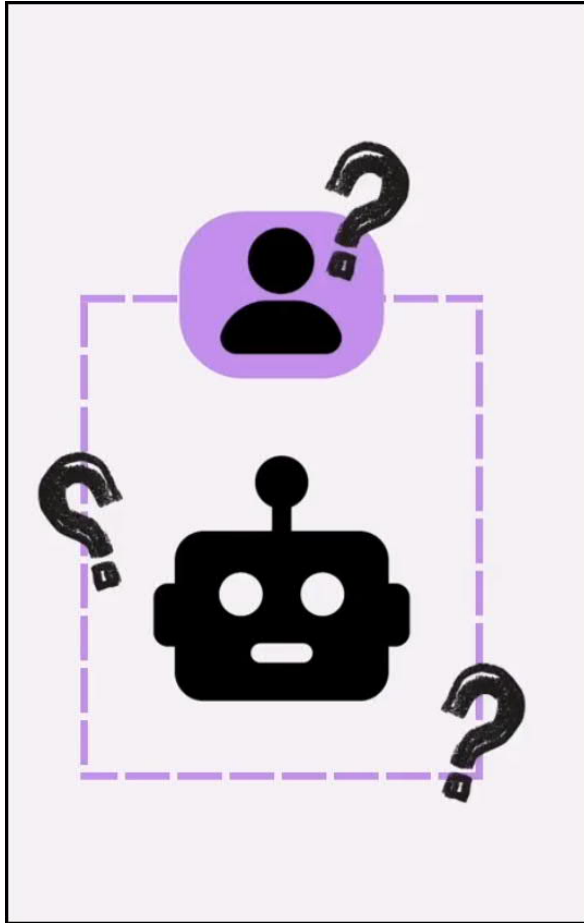
HUMAN IN THE LOOP

Human-in-the-loop is useful for the following:

- High-stakes operations requiring human approval (e.g. database writes, financial transactions).
- Very common in VSCode Agent-based tasks in Copilot chat
- Compliance workflows where human oversight is mandatory.
- Long-running conversations where human feedback guides the agent.

Human-In-the-loop Middleware pauses agent execution for human approval, editing, or rejection of tool calls before they execute.

HUMAN IN THE LOOP MIDDLEWARE



HUMAN IN THE LOOP MIDDLEWARE

When a model proposes an action that might require review—for example, writing to a file or executing SQL—the middleware can pause execution and wait for a decision.

It does this by checking each tool call against a configurable policy.

- If intervention is needed, the middleware issues an interrupt that halts execution.
- The graph state is saved using LangGraph's persistence layer, so execution can pause safely and resume later.

A human decision then determines what happens next: the action can be approved as-is (**approve**), modified before running (**edit**), or rejected with feedback (**reject**).

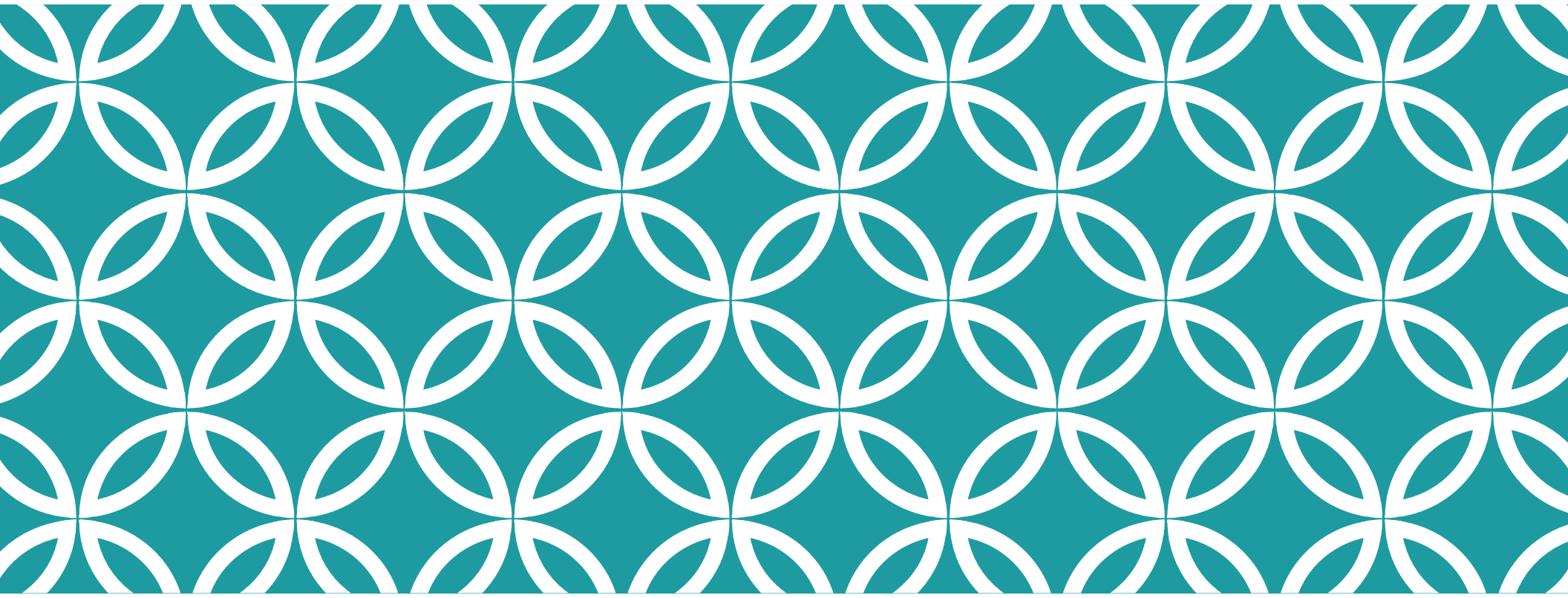
INTERRUPT DECISION TYPES

Decision Type	Description	Example Use Case
 approve	The action is approved as-is and executed without changes.	Send an email draft exactly as written
 edit	The tool call is executed with modifications.	Change the recipient before sending an email
 reject	The tool call is rejected, with an explanation added to the conversation.	Reject an email draft and explain how to rewrite it

The available decision types for each tool depend on the policy you configure in `interrupt_on`.

HUMAN IN THE LOOP POLICIES

```
agent = create_agent(  
    model="gpt-4.1",  
    tools=[write_file_tool, execute_sql_tool, read_data_tool],  
    middleware=[  
        HumanInTheLoopMiddleware(  
            interrupt_on={  
                # All decisions (approve, edit, reject) allowed  
                "write_file": True,  
                # No editing allowed  
                "execute_sql": {"allowed_decisions": ["approve", "reject"]},  
                # Safe operation, no approval needed  
                "read_data": False,  
            },  
            description_prefix="Tool execution pending approval",  
        ),  
    ],  
    # In production, use a persistent checkpointer like AsyncPostgresSaver.  
    checkpointer=InMemorySaver(),  
)
```



DYNAMIC PROMPTS

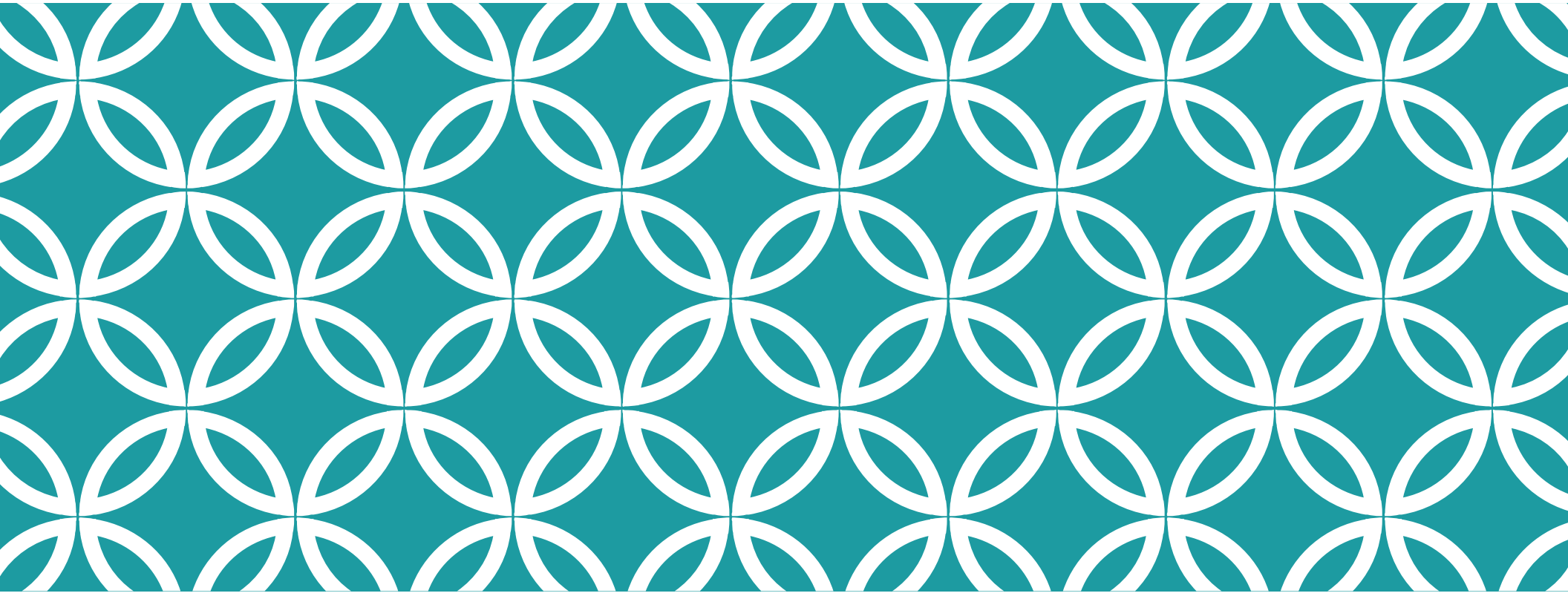


@dynamic_prompt decorator is a LCEL (LangChain Expression Language) feature that lets you define prompts as Python functions instead of manually constructing **PromptTemplate** or **ChatPromptTemplate**.

Creates runtime-adaptive prompts whose structure, messages, or content can change based on logic inside the function.

@dynamic_prompt turns a Python function into a prompt-producing runnable.

- Inputs → function arguments
- Output → prompt messages (or text)
- Logic → fully dynamic



DYNAMIC AGENTS



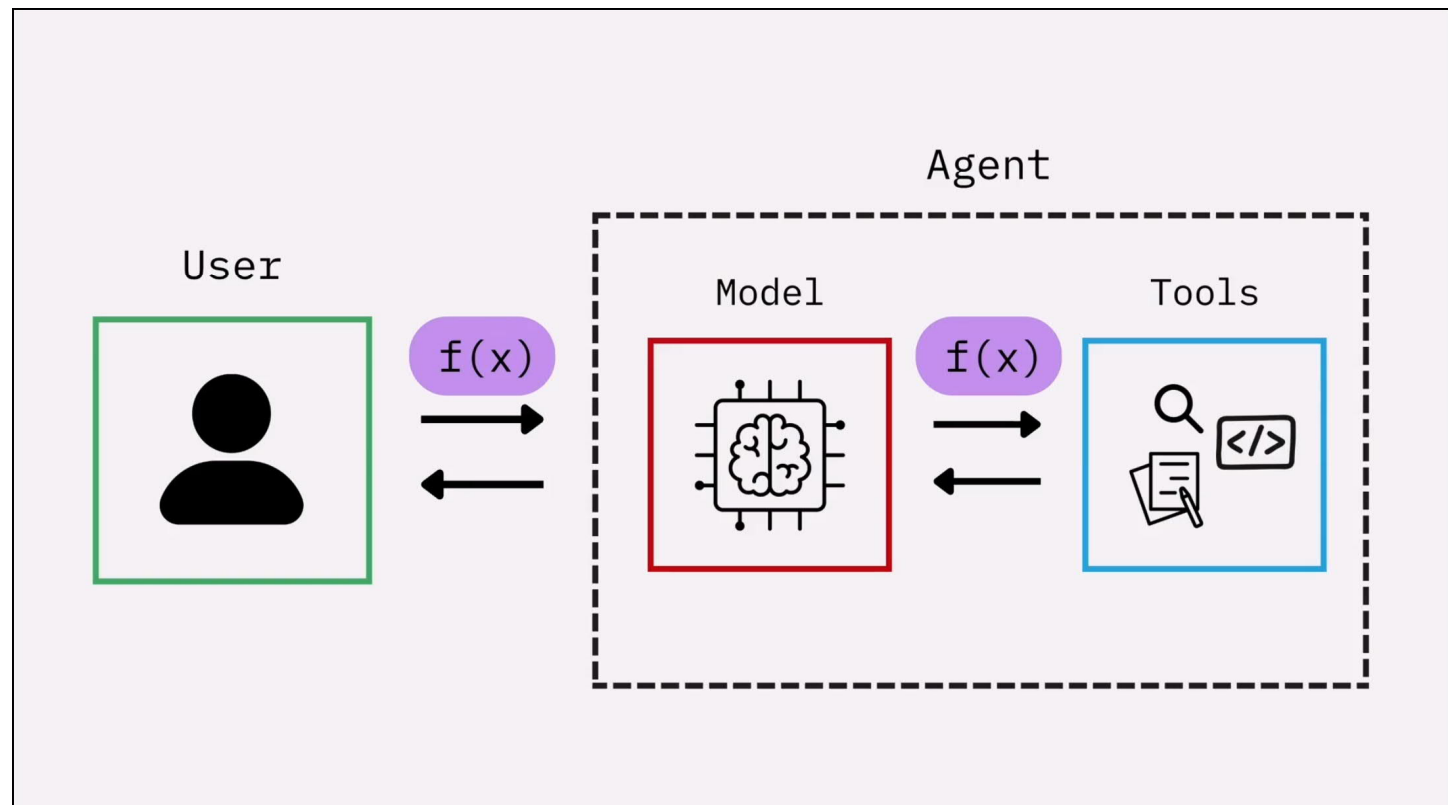
REAL WORLD SCENARIOS REQUIRE DYNAMIC FLOWS



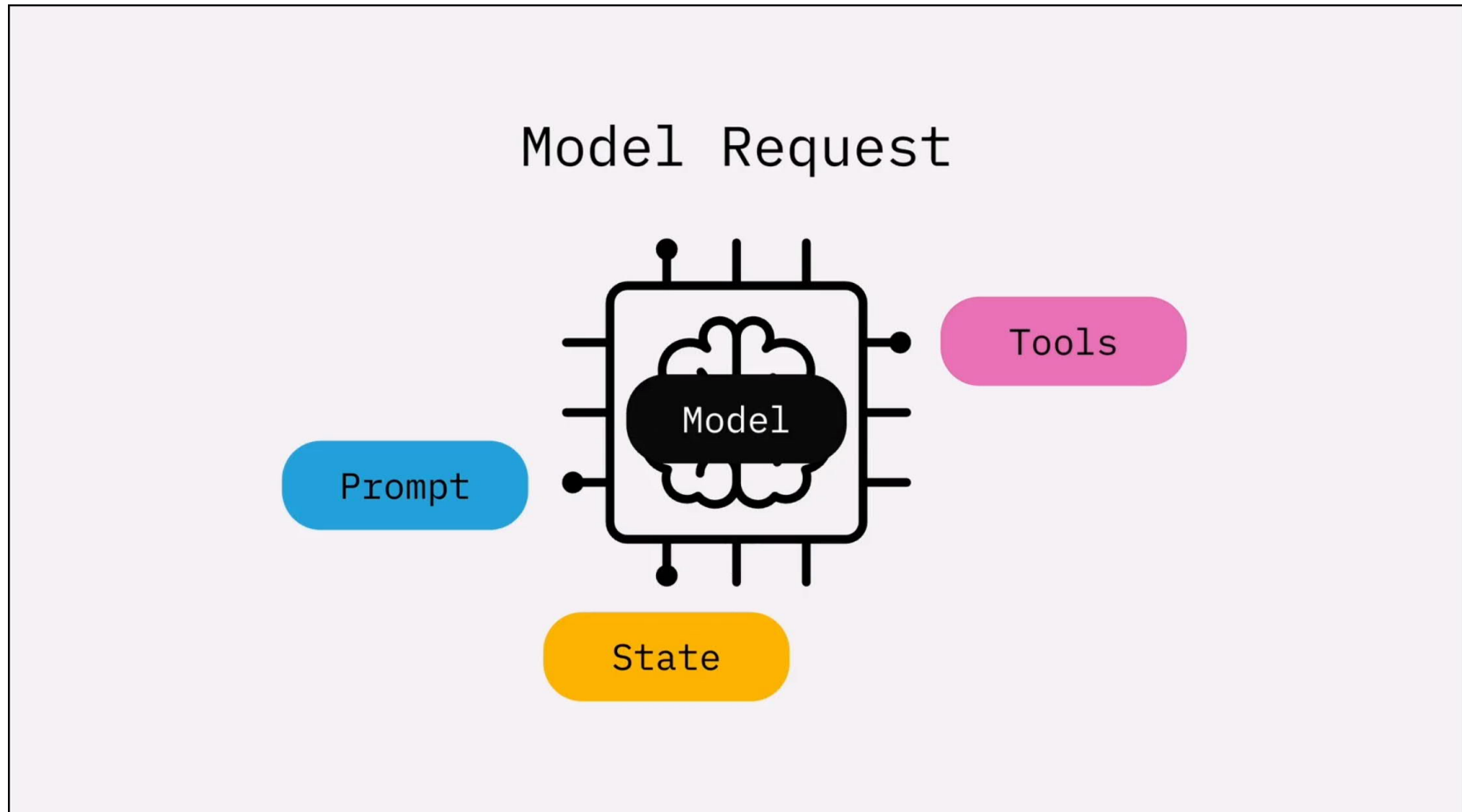
SO FAR — NODE STYLE MIDDLEWARE (NODE AS AN NODE.JS)

The “pipeline of nodes/runnables” is node-style middleware.

- Middleware sits *between* steps
- Can modify input or output
- Can log, validate, retry, timeout, etc.



MODEL REQUEST



DYNAMIC AGENTS — INTERCEPTING MODEL REQUEST

ModelRequest is a data structure within Langchain Middleware to intercept, inspect, or modify the request before it reaches the LLM

- **system_message/system_prompt**: The current system instructions for the model.
- **messages**: The full list of conversation messages being sent.
- **tools**: The list of tools available to the model for this specific call.
- **model_settings**: A dictionary for runtime overrides (e.g., temperature, max_tokens, or provider-specific headers).
- **state**: The current agent state, allowing you to modify the request based on previous turn data.
- **runtime**: Access to the execution context, including metadata and the Store (for persistent memory).

@WRAP_MODEL_CALL

Core utility designed to provide a consistent wrapper around the execution of Large Language Models (LLM)

Acts as a decorator that handles the "lifecycle" of a model interaction:

- Preprocessing: It prepares the input prompts and checks for any active Callbacks (e.g., logging, etc.)
- Dispatching: It triggers the `on_llm_start` event.
- Execution: It executes the actual model call (the "wrapped" function).
- Post-processing:
 - On Success: It captures the `LLMResult`, triggers `on_llm_end`, and returns the output.
 - On Error: It captures the exception, triggers `on_llm_error`, and re-raises the error so your code can handle it.

DESIGN GUIDANCE

Wrap-style Middleware	Node-style Middleware	Tool Calls
ModelRequest	State + Runtime	ToolRuntime
Adjust an agent's tools, prompt, model while running	Adjust an agent's state while running	Agent adjusts its own state or retrieves runtime context
e.g. Giving an agent access to tools if the user is internal	e.g. Trimming messages from long conversations	e.g. Updating the state with the username when provided to the agent